

ESP32 Wetter Station Teil 1

Anthony Timmel

20. März 2026

Inhaltsverzeichnis

I	Einführung	3
1	Pinarten des ESP32	5
1.1	GPIO	5
1.2	I2C	6
1.3	SPI	6
2	Der Aufbau des ESP32 auf dem Board	7
2.1	Versorgungs- & Datenplan des ESP32	7
2.2	Draufsicht auf den ESP32 in Betrieb	8
2.3	Nebenansicht auf den ESP32 in Betrieb	9
2.4	Programmierung des ESP32	9
2.4.1	Klassen	9
II	Aufgaben	11
3	Aufgabe 1	13
3.1	Anforderungen	13
3.2	Hardware	13
3.2.1	Bauelemente	13
3.2.2	Vernetzung der Bauelemente	14
3.3	Programmcode	15
4	Aufgabe 2	17
4.1	Anforderungen	17
4.1.1	Teil 1	17
4.1.2	Teil 2	17
4.2	Hardware	17
4.2.1	Bauelemente	17
4.2.2	Vernetzung der Bauteile	18
4.3	Programmcode	19
4.3.1	Programmcode für AM312	19
4.3.2	Programmcode für LM393	19
4.3.3	Überarbeitung des while-loops	20
5	Aufgabe 3	22
5.1	Anforderungen	22
5.2	Hardware	22

5.2.1	Bauelemente	22
5.3	Programmcode	23
III	Resultat	24

Teil I

Einführung

In dem ersten Teil dieser Dokumentation beschäftigen wir uns mit dem ersten Teil der Aufgabe aus dem Unterricht LF7. In den nächsten Seiten wird Ihnen das Projekt *Wetterstation ESP32* im ersten Teil schriftlich vorgestellt. Dabei werden wir uns zuerst die benötigten Hardwarekomponenten anschauen, danach folgt die Verkabelung dieser Komponenten mit dem *ESP32* & am Ende schauen wir uns die Umsetzung in dem Quelltext an.

Kapitel 1

Pinarten des ESP32

In diesem Projekt verwenden wir mehrere unterschiedliche so genannte **Pins**. Dies sind spezielle Arten von Anschlüssen, die natürlich auch unterschiedlichen Zwecken dienen. Hier werden wir auf eine Handvoll eingehen, die auch tatsächlich im Projekt genutzt werden. Dabei unterscheiden sich diese immer in der Art & Weise, wie Daten von der Komponente & dem *ESP32* übertragen werden.

1.1 GPIO

GPIO steht für **General Purpose Input/Output**. Dabei kann dieser entweder ein Eingang- oder Ausgangssignal repräsentieren. Es wird dazu genutzt, um mit weiteren einfachen Komponenten zu kommunizieren oder Steuern.

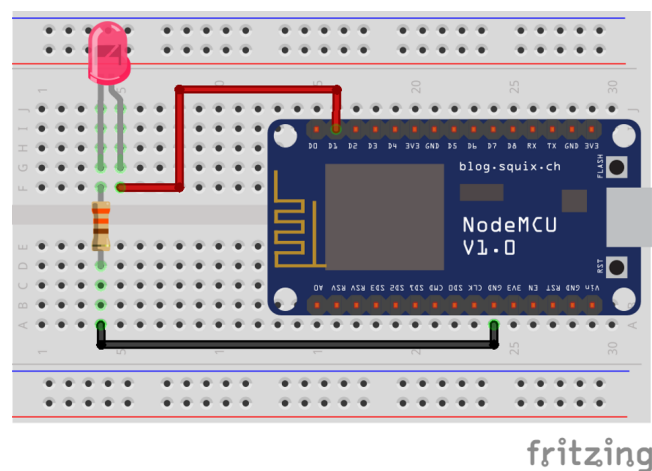


Abbildung 1.1: Dieses Bild zeigt den Aufbau von einem GPIO Port mit einer LED, die an den ESP32 angeschlossen ist.

1.2 I2C

I2C oder auch **Inter-Integrated-Circuit** ist ein serieller Kommunikationsbus, der 2 spezielle GPIO Pins benötigt **SDA** (Serial Data Line) & **SCL** (Serial Clock Line). Dabei wird *I2C* benötigt, um Daten schnell & in großer Form zu übertragen, sowie die Uhrzeit synchron über Komponenten hinweg zu halten.

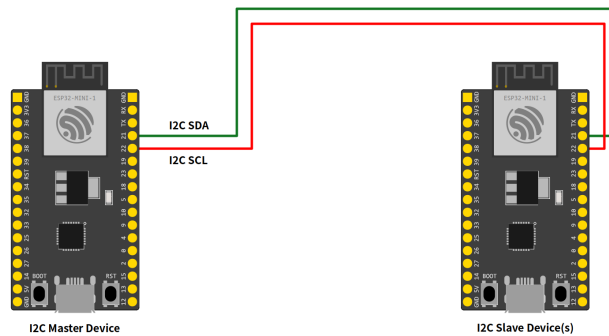
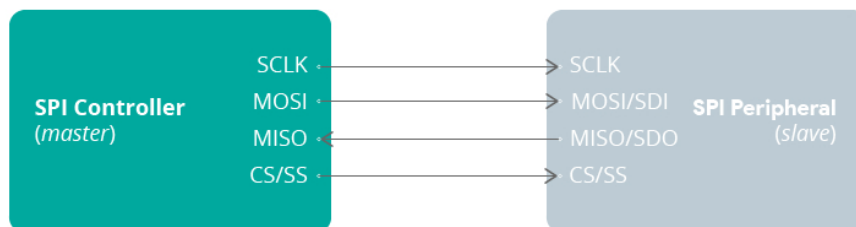


Abbildung 1.2: Dieses Bild zeigt den Aufbau von I2C. In diesem Bild sind 2 *ESP32* miteinander Verbunden, dabei können sie Daten schnell & einfach Übertragen. Diese Verbindung benötigt jedoch einen Verantwortlichen *auch Master genannt*. Jedes weitere Bauteil in dieser Verbindung ist ein Slave, dieser hört priorisiert auf die Anweisungen vom Master, kann jedoch, je nach Einstellungen, auch die gleichen Operationen wie der Master.

1.3 SPI

SPI steht für **Serial Peripheral Interface** und ist ein synchrones serielles Datenprotokoll, das von einem Mikrocontroller (Controller oder auch Master genannt) zur Kommunikation mit einem oder mehreren Peripheriegeräten (auch Slaves genannt) verwendet wird. Beispielsweise kommuniziert der ESP32-Board mit einem Sensor, der SPI unterstützt, oder mit einem anderen Mikrocontroller.

Abbildung 1.3: SPI Aufbau



Kapitel 2

Der Aufbau des ESP32 auf dem Board

2.1 Versorgungs- & Datenplan des ESP32

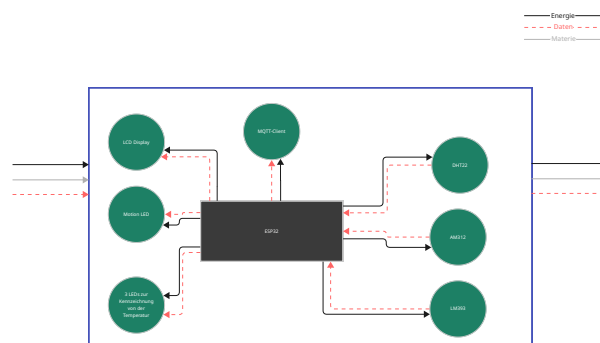


Abbildung 2.1: Aufbau des ESP32 auf dem Board. Diese Abbildung beinhaltet bereits Module, die nicht in dieser Dokumentation erklärt wurden. Diese Abbildung wurde dennoch zur Vollständigkeit nicht abgeändert. In der Dokumentation Teil 2, befindet sich dann die fehlenden Modulen

2.2 Draufsicht auf den ESP32 in Betrieb

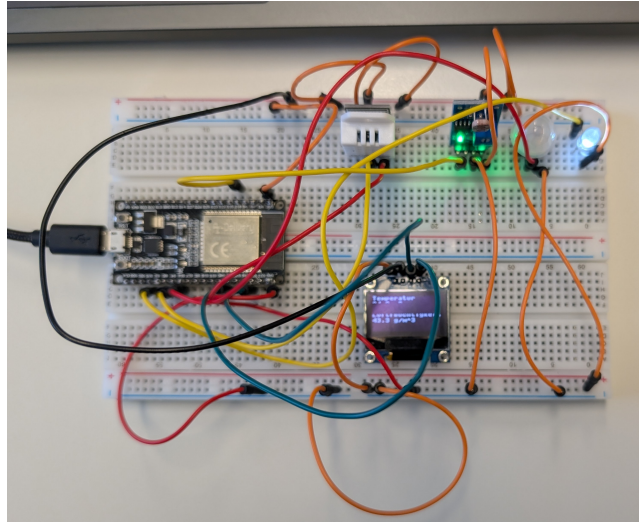


Abbildung 2.2: In dieser Draufsicht sehen Sie das fertig gebaute Produkt aus dem Projekt Teil 1. Die Bauteile sind hier bereits mit dem ESP32 verbunden & der benötigte Quelltext ist in einem kompletten funktionalen Zustand.

2.3 Nebenansicht auf den ESP32 in Betrieb

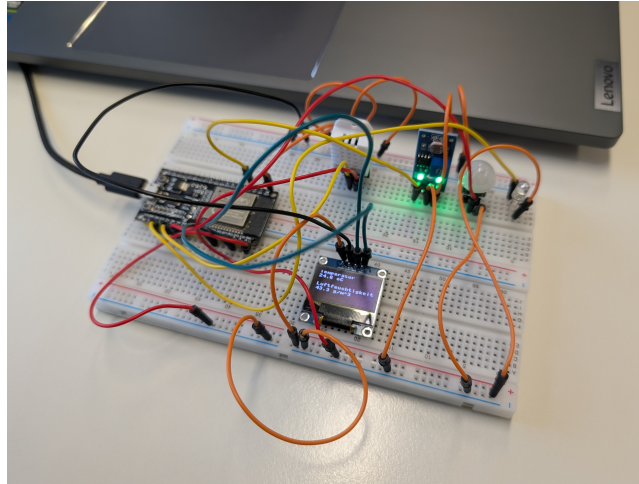


Abbildung 2.3: In dieser Nebenansicht sehen Sie das fertig gebaute Produkt aus dem Projekt Teil 1. Die Bauteile sind hier bereits mit dem ESP32 verbunden & der benötigte Quelltext ist in einem kompletten funktionalen Zustand.

2.4 Programmierung des ESP32

Damit der ESP32 & die anderen Komponenten miteinander kommunizieren können sowie die Weitergabe der Daten an externe Dienste, müssen diese Funktionen zum Teil manuell als Programmiercode geschrieben werden. Hierbei benutzen wir eine einfachere Variante, indem wir Python als Programmiersprache verwenden. Zusammen mit der Erweiterung für Microcontroller namens MicroPython können wir den ESP32 in einfachen Schritten anleiten, bestimmte Aufgaben zu erledigen.

2.4.1 Klassen

Bestimmte Klassen werden in den Code-Snippets immer wieder vorkommen. Dabei handelt es sich um wesentliche Grundfunktionen von MicroPython oder Python an sich.

Pin

Die `Pin` Klasse wird in MicroPython verwendet, um eine Schnittstelle zwischen dem ESP32 & der Komponente auf dem Steckbrett zu deklarieren. Eine Deklaration eines Pins sieht wie folgend aus:

```
1 PIN_VALUE = 1
2 PIN_IO = Pin.OUT
3 PIN_DRIVE = Pin.DRIVE_0
4 pin = Pin(PIN_VALUE, PIN_IO, drive=PIN_DRIVE)
5
```

1. `PIN_VALUE` => GPIO Nummer des Pins. Zusehen ist diese auf der Abbildung 2.4
2. `PIN_IO` => I/O Richtung des Pins. Hierbei wählt man zwischen `Pin.IN` (*Eingehend*) oder `Pin.OUT` (*Ausgehend*). Der Pin agiert dann ausschließlich in diese Richtung.
3. `PIN_DRIVE` => Die Ausgabe von der Stromstärke in 4 verschiedenen Stufen. (**Optional**)

Die offizielle Dokumentation befindet sich auf der [MicroPython](#) Seite. Dort befindet sich eine ausführliche Erklärung der verschiedenen Variablen für die Erstellung einer Instanz der Pin Klasse.

ESP32-DevKitC

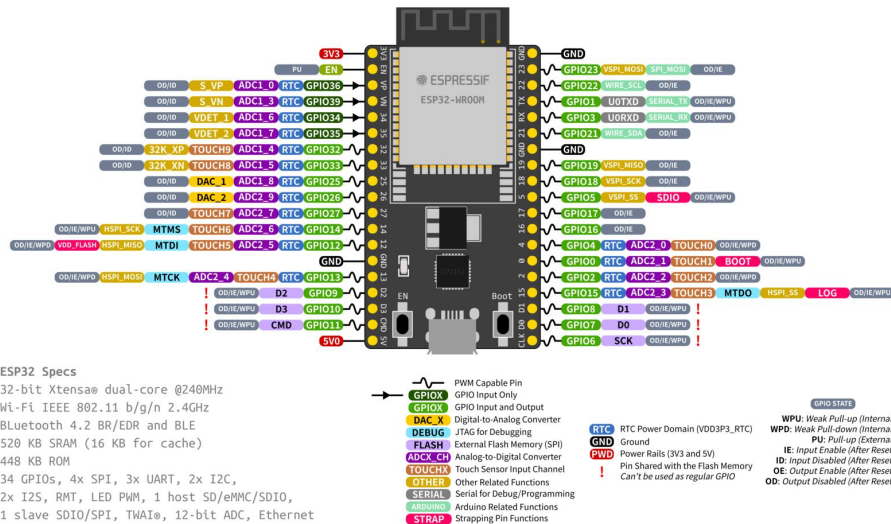


Abbildung 2.4: In dieser Abbildung sehen Sie die Verteilen der Pins auf dem ESP32. In den folgenden Tabellen innerhalb der Bauelemente Sektion, werden Referenzen zu dieser Übersicht gemacht.

Teil II

Aufgaben

In den folgenden Aufgaben werden wir uns an die Aufgabenbeschreibung *Projekt FI24_Teil 1.pdf* & *Projekt FI24_Teil 1_2.pdf* halten. Dabei wurde ebenfalls eine *Linkliste.pdf* Datei bereitgestellt, die bereits die benötigten Informationen für dieses Projekt beherbergt. Innerhalb der Aufgaben werden wir uns immer wieder mit diesen Dateien & deren Inhalten auseinandersetzen.

Kapitel 3

Aufgabe 1

3.1 Anforderungen

Bevor das cyber-physische System (CPS) in das **interne** Netzwerk integriert werden kann sollen Sie testen, ob die Möglichkeit besteht mit einem ESP32 als Grundlage des CPS, die Temperatur und die Luftfeuchtigkeit mit einem Sensor (**DHT22**) einzulesen und zu verarbeiten. Für die sekundliche Übermittlung der Daten an den später auszuwertenden MQTT-Client sollen die Sensorwerte in das Datenformat JSON abgelegt werden.

Zusatz: Greifen Sie auf den Inhalt des in MicroPython erstellten JSON-Strings zu und **geben** Sie die Sensordaten mit der **print**-Funktion im Terminal aus.

3.2 Die Vernetzung der Hardware auf dem Steckbrett

3.2.1 Bauelemente

ESP32

Der *ESP32* befindet sich bereits **vorinstalliert** auf dem Steckbrett und kann, durch das **Hinzufügen** des Stromversorgungs- & Datenübertragungskabels lässt sich die Einheit, von dem Computer aus Steuern.

Tabelle 3.1: Verteilung der Pin-Nutzung auf dem ESP32

Pin	Aufgabe
3V3	Stromversorgung des Bauteils
GND	Die Masse des ESP32
GPIO4	Universeller Eingangspin für die Messdaten es DHT22

DHT22

Das Erweiterungsmodul *DHT22* ist ein Modul, was es **ermöglicht**, die Raumtemperatur sowie die Luftfeuchtigkeit in der Umgebung **festzustellen**.

Tabelle 3.2: Verteilung der Pin-Nutzung auf dem DHT22

Pin	Aufgabe
VCC	Stromversorgung des Bauteils
GROUND	Die Masse des DHT22
DATA	Datenübertragungsport für die Messdaten es Bauteils

3.2.2 Vernetzung der Bauelemente

Der *ESP32* ist mit dem Pin **ADC2_0 GPIO4** mit dem *DHT22* **DATA** Pin verbunden. Die Stromversorgung des *DHT22* geht über den **VCC** Pin des Bauteils, dieser ist mit der Stromversorgung von dem **3V3** Pin des *ESP32* verbunden. Der **GROUND** Pin des Bauteils, ist mit der Masse **GND** des *ESP32* zusammengeschlossen, damit die Schaltung elektrotechnisch Funktionstüchtig werden kann.

3.3 Programmcode für den ESP32

```
1 dht_device = DHT22(Pin(4, Pin.IN))
2
```

Bevor die Daten aus dem Modul auslesen können, müssen wir zuerst die Verbindung zu diesem aufbauen. Dabei definieren wir das `dht_device` als eine Instanz aus der Klasse `DHT22`. Diese benötigt jedoch bei der Initialisierung eine Instanz der `Pin` Klasse. Hierbei müssen wir die

Mithilfe dieser Funktion, ist es uns möglich, die **Temperatur** und **Luftfeuchtigkeit** an dem **aktuellen Ort** zu bestimmen. Dafür sendet das Modul *DHT22* seine Daten an den Pin **G4**, diesen aben wir zuvor Pinverteilung ESP32 definiert.

Durch **DHT22** aus der Datei `dht` erhalten wir die Möglichkeit, Daten über eine Erweiterung des Protokolls abzugreifen. Dafür haben wir den Input-Pin der `DHT22` Klasse `DHT22(Pin(4, Pin.IN))` bei der Initialisierung **übergeben**. Im Anschluss ist es uns nun möglich, eine Messung **mithilfe** von `dht_device.measure()` können wir das Modul **auffordern**, eine Messung der Temperatur sowie Luftfeuchtigkeit vorzunehmen. Im Anschluss **erhalten** wir die Temperatur sowie Luftfeuchtigkeit über die jeweilige Methode des Objektes und geben diese am Ende der Funktion als `Pair` zurück.

```
1 while True:
2     temp, humid = get_device_temp_and_humid()
3
4     raw_json_object = {
5         "Temperatur": temp,
6         "Luftfeuchtigkeit": humid
7     }
8
9     json_object = json.dumps(raw_json_object)
10
11     print(json_object)
12     sleep(1)
13
```

Mithilfe **dieser** Schleife, ermöglicht es uns, bestimmte Bauteile **jede** Sekunde, auf dem *ESP32* abzufragen und im Anschluss, die Daten weiter zu verarbeiten. Dafür holen wir uns die **Temperatur & Luftfeuchtigkeit** über die Funktion `get_device_temp_and_humid()`. Im Anschluss erstellen wir ein `Dictionary`, um die Daten später im Programmcode leichter zu einem `JSON-String` zu konvertieren. Die Konvertierung zu einem `JSON-String` findet mit der Hilfe von `json` aus der Datei `json`. Dadurch, dass wir **bereits** einen `Dictionary` erstellt haben, können wir dies nun direkt der `dump` Funktion **übergeben**, und diese gibt uns einen vollständigen `JSON-String` zurück. Daraufhin geben wir diesen `JSON-String` in der `Console` aus und warten 1 Sekunde, bis die Schleife erneut durchläuft.

Der vollständige Programmcode für die Aufgabe Teil 1 sieht also wie folgt aus:

```
1  from machine import Pin
2  from dht import DHT22
3  from time import sleep
4  import json
5
6  def get_device_temp_and_humid():
7      dht_device = DHT22(Pin(4, Pin.IN))
8      dht_device.measure()
9      temp = dht_device.temperature()
10     humid = dht_device.humidity()
11
12     return temp, humid
13
14     while True:
15         temp, humid = get_device_temp_and_humid()
16
17         raw_json_object = {
18             "Temperatur": temp,
19             "Luftfeuchtigkeit": humid
20         }
21
22         json_object = json.dumps(raw_json_object)
23
24         print(json_object)
25         sleep(1)
26
```

Kapitel 4

Aufgabe 2

4.1 Anforderungen

4.1.1 Teil 1

Erweitern Sie das ESP32 um **einen** Bewegungsmelder und einen Fotowiderstand, dessen Daten Sie ebenfalls zu dem **bereits erstellten** JSON-Syntax hinzufügen. Das Ansprechen des Bewegungsmelders **soll** hierbei über eine LED signalisiert werden.

4.1.2 Teil 2

Ergänzen Sie Ihren Prototypen dahingehend, dass eine **grüne**, **gelbe** oder **rote** LED leuchten soll wenn die Temperatur $\geq 30\text{ °C}$ (rot), $\geq 25\text{ °C}$ und $< 30\text{ °C}$ (gelb) oder $\leq 20\text{ °C}$ (grün) beträgt

4.2 Hardware

4.2.1 Bauelemente

Inkrementeller Aufbau zu Kapitel 3.

ESP32

Tabelle 4.1: Verteilung der Pin-Nutzung bei dem ESP32

Pin	Aufgabe
VCC	Stromversorgung des Bauteils
GND	Die Masse des LM393
GPIO34	Universeller Analoger Eingangspin des ESP32 für den LM393
GPIO16	Universeller Eingangspin des ESP32 für den AM312

AM312

Das Erweiterungsmodul *AM312* ist ein Modul, was zur **Erfassung** von **Bewegungen** verwendet wird.

Tabelle 4.2: Verteilung der Pin-Nutzung bei dem AM312

Pin	Aufgabe
VCC	Stromversorgung des Bauteils
GND	Die Masse des AM312
VOUT	Datenübertragungsport für die Messdaten es Bauteils

LM393

Das Erweiterungsmodul *LM393* ist ein Modul, was zur **Erfassung** der **Helligkeit** verwendet wird. Das Erweiterungsmodul *LM393* ist ein Modul, was zur **Erfassung** der **Helligkeit** verwendet wird.

Tabelle 4.3: Verteilung der Pin-Nutzung bei dem AM312

Pin	Aufgabe
VCC	Stromversorgung des Bauteils
GND	Die Masse des LM393
AO	Analoger Datenübertragungsport für die Messdaten es Bauteils

4.2.2 Vernetzung der Bauteile

Der *ESP32* ist mit dem Pin **GPIO16** mit dem *AM312* **VOUT** Pin verbunden. Die Stromversorgung des *AM312* geht über den **VCC** Pin des Bauteils, dieser ist mit der Stromversorgung von dem **3V3** Pin des *ESP32* verbunden. Der **GND** Pin des Bauteils, ist mit der Masse **GND** des *ESP32* zusammengeschlossen, damit die Schaltung elektrotechnisch Funktionstüchtig werden kann.

Der *ESP32* ist mit dem Pin **ADC1_6 GPIO34** mit dem *LM393* **AO** Pin verbunden. Die Stromversorgung des *LM393* geht über den **VCC** Pin des Bauteils, dieser ist mit der Stromversorgung von dem **3V3** Pin des *ESP32* verbunden. Der **GND** Pin des Bauteils, ist mit der Masse **GND** des *ESP32* zusammengeschlossen, damit die Schaltung elektrotechnisch Funktionstüchtig werden kann.

4.3 Programmcode

4.3.1 Programmcode für AM312

```
1 def get_motion():
2     pir_pin = Pin(16, Pin.IN)
3     return bool(pir_pin.value())
4
```

Diese Funktion gibt den **aktuellen** Wert (True/False) zurück, ob der Bewegungsmelder *AM312*, eine **Bewegung** innerhalb von 100° zwischen 3-5 Metern **wahrgenommen** hat. Sollte **keine** Bewegung wahrgenommen sein, gibt dieser ein False zurück. Die Methode `pir_pin.value()` gibt jedoch eine Zahl (**1** oder **0**) zurück. Damit diese jedoch **besser Nachvollziehbar** im Programmcode ist, wird diese zu einem Boolean **gecastet**. Dadurch wird **sichergestellt**, dass die Erkennung von einer Bewegung **nur 2** unterschiedliche Zustände haben kann.

Für die LED definieren wir außerhalb einer Funktion folgendes:

```
1 led = Pin(15, Pin.OUT, drive=Pin.DRIVE_0)
2
```

Hierbei **definieren** wir den **GPIO15** Pin, als Ausgangspin mit einem **maximalen** Vorwiderstand, damit wir das LED Bauteil nicht beschädigen.

4.3.2 Programmcode für LM393

```
1 def get_luminance(gamma=0.7, rl10_kohm=50,
2     fixed_resistor_ohm=2000):
3
4     pin = ADC(Pin(34, Pin.IN))
5     pin.atten(ADC.ATTN_11DB)
6
7     V_REF = 3.3
8     MAX_ADC = 4095
9
10    raw_value = pin.read()
11
12    voltage_out = raw_value / MAX_ADC * V_REF
13
14    if voltage_out == 0:
15        return 0
16
17    resistance_ldr = fixed_resistor_ohm * (V_REF -
18        voltage_out) / voltage_out
19
20    rl10_ohm = rl10_kohm * 1000
21
22    basis = (rl10_ohm * pow(10, gamma) / resistance_ldr)
23    exponent = 1 / gamma
24
25    return pow(basis, exponent)
```

Zuerst richten wir den Pin für den *LM393* ein, indem wir die spezielle Klasse `ADC` dafür importieren und die Pinzuweisung dieser Klasse, im Constructor direkt übergeben. Im nächsten Schritt, erweitern wir den Messbereich des Analog-Digital-Converters (*ADC*) auf **3.3V** durch die Konstante `ADC.ATTN_11DB`. Sollte diese Einstellung nicht vorgenommen werden, wäre die *ADC* Spannung bis ca. **1.0V** präzise messbar.

Im Anschluss definieren wir 2 Konstanten für die Widerstandsberechnung & Umrechnung der Spannung (Volt).

`V_REF` ist hierbei die Referenzspannung von 3.3V, die das Erweiterungsmodul von dem ESP32 als Stromversorgung erhält.

`MAX_ADC` ist die Auflösung des analogen Signals. Dies wird hierbei auf **12-Bit** Auflösung gesetzt ($2^{21} - 1$).

`raw_value` erhält nun die analogen Daten des *LM393*.

`voltage_out` ist die korrekte Umrechnung des *12-Bit-Wertes* in Spannung (Volt). Bevor wir mit der Widerstandrechnung anfangen dürfen, überprüfen wir, ob die Ausgehende Spannung `voltage_out` gleich 0 ist, wenn ja, geben wir direkt den Lux Wert 0 zurück, da bei einer ausgehenden Spannung von 0V kein Licht wahrgenommen werden kann, oder wird.

Nun erfolgt die Widerstandsrechnung ($R_LDR = resistance_ldr$), dabei ist die Formel wie folgt $R_LDR = R_fix * \frac{V_REF - V_out}{V_out}$.

Nach der Widerstandsrechnung, müssen wir, bevor wir die Lux berechnen können, die kOhm in Ohm umrechnen. Dafür rechnen wir die kOhm `r110_kohm` mit `*1000` und erhalten die `r110_ohm`.

Jetzt ist der Weg frei, die Lux zu berechnen. Jedoch benötigen wir erst die Basis, das ist mit folgender Formel errechenbar. ($R110 * 10^{gamma} / R_LDR$)

Im Anschluss müssen wir den Exponenten berechnen, dies erfolgt über eine einfache Division $exponent = 1/gamma$.

Der letzte Schritt, berechnen wir nun die Lux, indem wir die `basis` hoch `exponent` rechnen, das Ergebnis geben wir dann zurück.

4.3.3 Überarbeitung des while-loops

Im Anschluss müssen wir den While-Loop mit den neuen Funktionen ergänzen. Die vollständige neue While-Schleife sieht nun wie folgt aus:

```

1  while True:
2      temp, humid = get_device_temp_and_humid()
3      motion = get_motion()
4
5      luminance = get_luminance()
6
7      if motion:
8          led.value(1)
9      else:
10         led.value(0)
11
12     raw_json_object = {
13         "Temperatur": temp,
14         "Luftfeuchtigkeit": humid,
15         "Bewegung": motion,
16         "Helligkeit": luminance

```

```

17     }
18
19     json_object = json.dumps(raw_json_object)
20
21     print(json_object)
22     sleep(1)
23

```

Wir haben die While-Schleife mit den `motion` & `luminance` variable aus den beiden bereits **vordefinierten** Funktionen erweitert. Damit jedoch nun, wie gefordert, auch die LED sich **updated**, sollte jemand **in den Bereich** des Sensor *AM321* kommen, muss die variable `motion` nun in einer **if-else** Bedingung **modifiziert** werden. Dafür setzen wir den LED-Pin auf 1 sollte Bewegung **wahrgenommen** werden, trifft dies **nicht** mehr zu, setzen wir 0, dadurch leuchtet die LED nicht mehr. Der Effekt bei `value()` ist, dass der Pin *GPIO16* entweder mit Spannung versorgt wird, oder nicht.

Kapitel 5

Aufgabe 3

5.1 Anforderungen

Es **soll** die Möglichkeit bestehen die Temperatur und die Luftfeuchtigkeit vor Ort auf **einem OLED Display** (SSD1306) visualisieren zu können

5.2 Hardware

5.2.1 Bauelemente

Inkrementeller Aufbau zu der Teilaufgabe 2.

ESP32

Tabelle 5.1: Verteilung der Pin-Nutzung bei dem ESP32

Pin	Aufgabe
VCC	Stromversorgung des Bauteils
GND	Die Masse des ESP32
GPIO21 (WIRE_SDA)	Universeller Ein-/Ausgangspin Fähigkeit für SDA
GPIO22 (WIRE_SCL)	Universeller Ein-/Ausgangspin Fähigkeit für SCL

AM312

Das Erweiterungsmodul *SSD1306* ist ein Modul, was zum **Anzeigen** von **Informationen** (LCD-Monitor) genutzt wird.

Vernetzung der Bauteile

Der *ESP32* ist mit dem Pin **GPIO21** mit dem *SSD1306* **SDA** Pin und der **GPIO22** mit dem *SSD1306* **SCL** Pin verbunden. Die Stromversorgung des *SSD1306* geht über den **3.3V** Pin des Bauteils, dieser ist mit der Stromversorgung von dem **3V3** Pin des *ESP32* verbunden. Der **GND** Pin des Bauteils,

Tabelle 5.2: Verteilung der Pin-Nutzung bei dem SSD1306

Pin	Aufgabe
3.3V	Stromversorgung des Bauteils
GND	Die Masse des SSD1306
SDA	Serial-Dataline für die Datenübertragung zum LCD-Display
SCL	Serial-Clock für die In-Sync Datenübertragung

ist mit der Masse **GND** des *ESP32* zusammengeschlossen, damit die Schaltung elektrotechnisch Funktionstüchtig werden kann.

5.3 Programmcode

```

1  pin = SoftI2C(sda=Pin(21), scl=Pin(22))
2  display = ssd1306.SSD1306_I2C(128, 64, pin)
3

```

`pin` wird hier wie zuvor, mit der **Wrapper**-Klasse `SoftI2C` erzeugt. Dabei wird der `sda` und `scl` Pin definiert. Die instanziierte Klasse, kann wird nun **erneut** wieder in eine Wrapper-Klasse `SSD1306_I2C` übergeben. Dadurch erlangen wir die Möglichkeit, das LCD-Display *SSD1306* ansprechen.

Die `SSD1306` Klasse ist **keine** vordefinierte Klasse aus einem der MicroPython Dateien, sondern diese ist **manuell** auf dem *ESP32* hinterlegt. Der Programmcode für **diese** Datei ist aufgrund der Größe **nicht** in das Dokument implementiert.

Der Programmcode zur Erweiterung der `SoftI2C`-Klasse findest du [hier \(Github\)](#).

[Tutorial \(Random Nerd Tutorials\)](#) zur Einrichtung der Erweiterungsklasse.

Nachdem die `SSD1306`-Klasse nun verfügbar ist, kann die Funktion `display_text()` erstellt werden.

```

1  def display_text(temp, humid):
2      display.fill(0)
3      display.text("Temperatur", 0, 0)
4      display.text(str(temp) + " Grad", 0, 10)
5      display.text("Luftfeuchtigkeit", 0, 30)
6      display.text(str(humid) + " g/m^3", 0, 40)
7      display.show()
8

```

Damit sich der Text, bei einem Update **nicht** überlappt, muss zuvor die Fläche **zurückgesetzt** werden. Das Zurücksetzen der Anzeigefläche passiert, indem wir eine schwarze Fläche `display.fill(0)` zeichnen. Im Anschluss schreiben wir die **Temperatur & Luftfeuchtigkeit**, mit einem Versatz, auf den LCD-Monitor und aktualisieren die Anzeige mit `display.show()`.

Teil III

Resultat

Der *ESP32* fungiert nun als zentrale Einheit zum Messen von Temperatur, Luftfeuchtigkeit & Helligkeit. Ebenfalls nimmt er Bewegungen war & gibt diese anhand einer LED-Lampe (*In unserem Aufbau, eine weiße LED*) wider. Die Temperatur & Luftfeuchtigkeit wird auf einem kleinen LCD-Bildschirm mithilfe von einer I2C Verbindung an diesen geschickt.